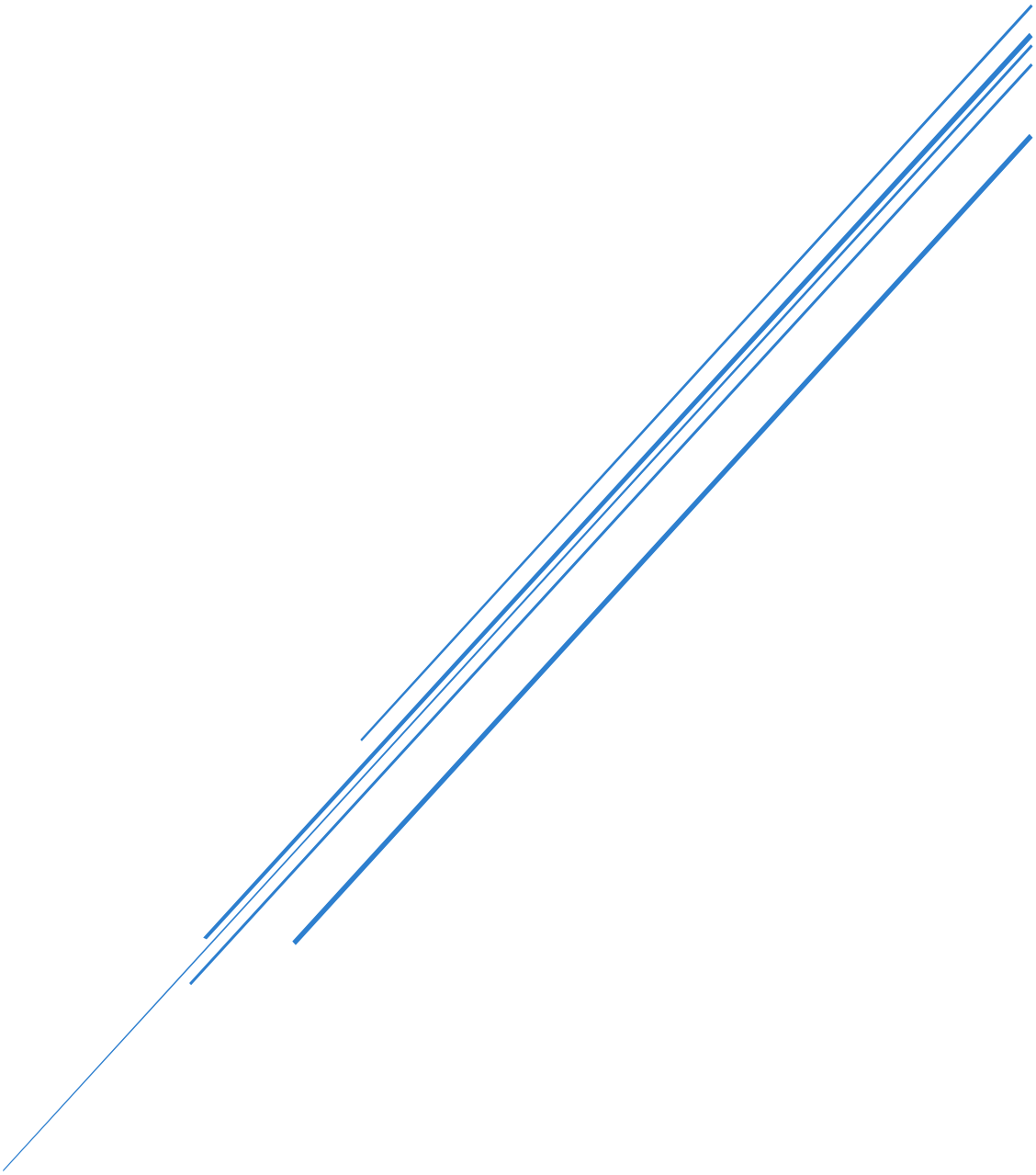
LA GUARDIA AIRPORT

Memoria completa del proyecto LaGuardia



Manuel González Quesada

1. Contexto y Justificación del Proyecto

1.1. El accidente del vuelo AC8646 como punto de partida

La seguridad operacional en el entorno aeroportuario constituye uno de los mayores retos de la ingeniería civil y aeronáutica moderna, especialmente en aeródromos con alta densidad de tráfico y limitaciones geográficas. Un caso práctico es el Aeropuerto Internacional LaGuardia (LGA) en Nueva York, cuya configuración de pistas cruzadas (04/22 y 13/31) exige una coordinación milimétrica entre el control de tráfico aéreo (ATC), las aeronaves y los vehículos de superficie.

El presente proyecto toma como eje y caso de estudio el grave accidente ocurrido en la madrugada del 22 de marzo de 2026 en dichas instalaciones. A las 23:37 horas, el vuelo AC8646 de Air Canada Express (operado por un avión regional Bombardier CRJ-900 procedente de Montreal) colisionó durante la maniobra de aterrizaje en la pista 04 contra el camión de extinción de incendios aeroportuario (ARFF) Rescue 35 (un Oshkosh Striker 1500), el cual se encontraba cruzando la pista activa para atender una emergencia no relacionada en el extremo opuesto del aeropuerto. El impacto provocó la destrucción de la sección delantera de la aeronave, causando el fallecimiento de ambos pilotos y dejando un balance de 39 heridos.

1.2. Análisis del fallo sistémico en la vigilancia terrestre

De acuerdo con las investigaciones de la Junta Nacional de Seguridad en el Transporte (NTSB), esta tragedia no fue producto de un fallo aislado, sino de la alineación de múltiples deficiencias sistémicas (el denominado "modelo del queso suizo" en seguridad aérea):

El vehículo de bomberos carecía de un transpondedor activo que permitiera su identificación inequívoca en las pantallas de radar de la torre de control.

Aunque LaGuardia cuenta con el avanzado sistema de detección de movimiento en superficie ASDE-X (que fusiona radar, multilateración y ADS-B), su lógica de seguridad (Safety Logic) fue incapaz de generar una alerta preventiva. La proximidad de varios vehículos de emergencia moviéndose simultáneamente impidió al algoritmo consolidar una traza de seguimiento fiable.

El siniestro se produjo bajo condiciones meteorológicas adversas (lluvia intensa y visibilidad reducida a 4,8 km) y con una dotación reducida en el turno de noche de la torre de control, donde cada controlador gestionaba dos posiciones de forma simultánea. Cuando el controlador se percató del conflicto y ordenó detenerse al camión mediante radiofrecuencia, el margen de tiempo resultó insuficiente para evitar la colisión.

1.3. Justificación: La necesidad de un sistema de alerta directa en cabina

El análisis del siniestro del vuelo AC8646 pone de manifiesto una vulnerabilidad crítica en la aviación comercial actual: la dependencia casi exclusiva de los sistemas de vigilancia en tierra y de la intervención humana del controlador para garantizar que una pista está libre o que una aproximación es segura en los metros finales.

Cuando la visibilidad exterior es nula y los radares de superficie fallan al detectar un obstáculo, la tripulación de vuelo carece de herramientas autónomas en cabina que les adviertan del peligro inminente en tiempo real.

Entre las medidas de seguridad propuestas tras el accidente, destaca el desarrollo e implementación de sistemas de alerta directa a bordo de la aeronave, capaces de monitorizar de forma autónoma el estado del avión y su proximidad al suelo u obstáculos durante las fases críticas de aterrizaje y despegue, funcionando como una red de seguridad independiente de la torre de control.

Con el objetivo de materializar una solución práctica a esta problemática, se ha diseñado e implementado un sistema de telemetría de vuelo e instrumentación embarcada en tiempo real sobre una maqueta de una aeronave.

A través de la integración de sensores inerciales, medidores ultrasónicos de altitud y protocolos de comunicación IoT (MQTT y Node-RED), el prototipo emula el comportamiento de un sistema moderno de aviónica y advertencia de proximidad a tierra (GPWS), dotando al "piloto" de un horizonte artificial telemétrico y un sistema automatizado de alertas por voz (Callouts de altitud y avisos de inclinación crítica o Bank Angle) que mejoran drásticamente la conciencia situacional durante la aproximación a pista.

2. Objetivos y Alcance del Proyecto

2.1. Objetivos Generales del Marco Teórico

El planteamiento original del proyecto de investigación sobre el aeropuerto internacional LaGuardia (LGA) y el siniestro del vuelo AC8646 se estructuró en torno a seis objetivos principales orientados a comprender de forma integral la seguridad aeroportuaria:

Objetivo 1 (Recreación de infraestructura): Reproducir a escala y con la máxima fidelidad posible la distribución física del aeropuerto LaGuardia, prestando especial atención a su configuración de pistas cruzadas (4/22 y 13/31), calles de rodaje (taxiways) y plataformas de estacionamiento.

Objetivo 2 (Señalización e iluminación): Comprender y exponer el funcionamiento de los sistemas de señalización horizontal (marcas de umbral, zonas de toma de contacto, barras de parada) y vertical/luminosa (luces de borde, sistema PAPI, ALS) que guían a las aeronaves tanto en horario diurno como nocturno o bajo baja visibilidad.

Objetivo 3 (Análisis de vigilancia de superficie): Estudiar la arquitectura del radar de superficie ASDE-X y su lógica de seguridad (Safety Logic), identificando sus capacidades reales de rastreo y las limitaciones críticas que presenta cuando operan en pista vehículos sin transpondedor en escenarios de alta densidad.

Objetivo 4 (Procedimientos aeronáuticos OACI): Analizar los protocolos estandarizados de la Organización de Aviación Civil Internacional para las fases de aproximación y aterrizaje, abarcando desde la fraseología de control aéreo y la gestión de recursos en cabina (CRM) hasta el aterrizaje por instrumentos mediante el sistema ILS.

Objetivo 5 (Implementación de sensorización IoT): Desarrollar un prototipo electrónico basado en microcontroladores capaz de monitorizar en tiempo real las variables físicas de la operación y emitir alertas preventivas automáticas ante situaciones de riesgo en la fase de aterrizaje o despegue.

Objetivo 6 (Propuestas de seguridad operacional): Formular medidas correctivas y mejoras tecnológicas viables —como la obligatoriedad de transpondedores en vehículos terrestres ARFF o la instalación de sistemas de alerta directa en cabina— basadas en las investigaciones de la NTSB para evitar futuras incursiones en pista.

2.2. Alcance Real del Desarrollo Práctico (Delimitación del Prototipo)

Debido a la envergadura técnica que supone recrear e instrumentar un complejo aeroportuario completo, ha sido necesario delimitar de forma estricta el alcance de fabricación física y electrónica para esta fase de entrega, separando el estudio teórico de la ejecución práctica.

Mientras que los Objetivos 2, 3, 4 y 6 han quedado plenamente cubiertos en la fase de investigación documental, el desarrollo físico y tecnológico (Objetivos 1 y 5) se ha acotado de manera exclusiva a la unidad aérea (la aeronave y su aviónica embarcada), descartando la construcción de la infraestructura terrestre.

De este modo, los límites de lo construido en el proyecto se definen claramente en dos bloques:

1. Elementos excluidos del desarrollo físico (Infraestructura Terrestre):

No se ha construido la base física del terreno aeroportuario, ni se han representado físicamente las pistas cruzadas 4/22 y 13/31 ni las calles de rodaje.

No se han maquetado las terminales de pasajeros, la torre de control ni el camión de bomberos (ARFF) involucrado en la colisión.

Al no existir una pista física, se ha descartado el montaje de fotorresistencias (LDR) empotradas en el pavimento que se planteaba inicialmente para emular los radares terrestres.

2. Elementos ejecutados y completados al 100% (Sistema Embarcado en el Avión):

Todo el esfuerzo de ingeniería electrónica, diseño mecánico e impresión 3D de soportes a medida se ha concentrado en el propio avión.

Se ha integrado con éxito en el interior de la maqueta un microcontrolador ESP32 conectado a un sensor inercial de 6 ejes (MPU6050) y a un sensor ultrasónico de proximidad (HC-SR04), dotando a la aeronave de conexión inalámbrica autónoma mediante portal cautivo (WiFiManager) y transmisión de datos por protocolo MQTT.

Se ha desarrollado por completo el servidor lógico capaz de procesar la física del avión en tiempo real. El alcance logrado incluye la estabilización matemática de los ángulos de vuelo (cabeceo, alabeo y guiñada), un sistema de calibración en reposo (Tara), un Horizonte Artificial dinámico y un sistema de avisos por voz (GPWS) que canta la altitud respecto al suelo y dispara alarmas sonoras ante inclinaciones críticas de las alas (Bank Angle).

En conclusión, el proyecto ha girado desde una maqueta arquitectónica de suelo hacia un prototipo funcional de aviónica e instrumentación de cabina, materializando de forma directa una de las principales soluciones propuestas tras el accidente de LaGuardia: dotar a los pilotos de sistemas de alerta autónomos a bordo del propio avión.

3. Tecnología Utilizada (Hardware, Software y Protocolos)

Para materializar el prototipo de telemetría embarcada y el simulador de cabina en tiempo real, se ha seleccionado un ecosistema tecnológico propio del Internet de las Cosas (IoT) industrial, dividiendo la arquitectura entre los componentes físicos instalados en el avión (Hardware) y las herramientas de procesamiento y comunicación en el servidor (Software y Protocolos).

3.1. Componentes Físicos (Hardware)

A continuación, se detallan los materiales electrónicos y mecánicos integrados en el interior de la maqueta de la aeronave:

Microcontrolador ESP32 (Placa de desarrollo NodeMCU): Se ha utilizado como el cerebro central a bordo del avión.

A diferencia de las placas Arduino tradicionales, el ESP32 cuenta con un procesador de doble núcleo de 32 bits y un módulo de radiofrecuencia WiFi y Bluetooth integrado en la propia placa.

Su función es alimentar a los sensores, leer sus señales eléctricas varias veces por segundo, empaquetar las lecturas y transmitirlas de forma inalámbrica hacia el ordenador sin necesidad de cables de datos que limiten el movimiento físico de la maqueta.

Sensor Inercial MPU6050 (Módulo comercial GY-521): Es una Unidad de Medición Inercial (IMU) microelectromecánica de 6 grados de libertad (6-DOF) que integra en un mismo chip un acelerómetro de 3 ejes y un giroscopio de 3 ejes.

Se comunica con el ESP32 a través del bus digital I2C (pines SDA y SCL). El acelerómetro mide la incidencia de la gravedad terrestre en los ejes X, Y y Z para calcular la inclinación estática del avión (cabeceo y alabeo), mientras que el giroscopio mide la velocidad angular de rotación para determinar el giro del morro (guiñada).

Sensor de Ultrasonidos HC-SR04: Es un módulo de telemetría acústica utilizado para simular el radioaltímetro real de una aeronave.

Funciona mediante dos pines digitales (TRIG en el pin 5 y ECHO en el pin 18): el sensor emite un pulso ultrasónico inaudible de 40 kHz hacia la mesa o el suelo y mide los microsegundos que tarda la onda en rebotar y regresar.

Aplicando la constante de la velocidad del sonido en el aire, permite calcular la altitud exacta de la maqueta al milímetro durante la maniobra de aterrizaje.

Cableado de interconexión (Cables Dupont / Jumpers) y Alimentación USB: Se han empleado latiguillos de conexión rápida hembra-hembra para enlazar los pines GPIO de la placa ESP32 con los módulos sensores, distribuyendo las líneas de alimentación de 3.3V / 5V, tierra común (GND) y canales de datos.

Para alimentar todo el conjunto electrónico, se utiliza un cable USB conectado a una fuente de alimentación de 5V.

Soporte estructural a medida (Impresión 3D FDM): Para evitar que el sensor MPU6050 se mueva dentro del avión o que las soldaduras inferiores de sus pines inclinen la placa y falseen las lecturas del horizonte artificial, se ha modelado en CAD e impreso mediante fabricación aditiva (FDM) un chasis de sujeción a medida.

Esta pieza cuenta con las dimensiones exactas del módulo GY-521 (20.8 mm x 15.8 mm), tolerancias de ajuste por fricción (+0.20 mm) y un cajeado inferior que garantiza que el sensor descanse totalmente plano y alineado con el eje longitudinal de la aeronave.

3.2. Herramientas de Programación y Comunicaciones (Software y Protocolos)

El flujo lógico de los datos, desde que se capturan en el aire hasta que se transforman en instrumentos visuales y alertas de voz, se apoya en cuatro pilares de software:

Entorno de Programación del Microcontrolador (Arduino IDE y C++):

La programación física de la placa ESP32 se ha realizado en lenguaje C/C++ utilizando el entorno de desarrollo oficial Arduino IDE.

El código se estructura en dos bloques fundamentales: una función de arranque (setup), donde se inicializa la comunicación por puerto serie a 115200 baudios, se configura la sensibilidad del MPU6050 (rango de 8G para aceleración, 500°/s para giro y filtro paso bajo de 21 Hz contra vibraciones) y se preparan los pines del ultrasonido; y un bucle infinito (loop) que se ejecuta cada 100 milisegundos (tasa de refresco de 10 Hz).

Para lograr esto, se han integrado librerías específicas como **Adafruit_MPU6050.h** (lectura de física inercial) y **PubSubClient.h** (empaquetado y envío de datos por red).

Gestión Dinámica de Red (Librería WiFiManager):

En lugar de escribir el nombre del WiFi y la contraseña de forma fija en el código C++ (lo que obligaría a enchufar el avión al ordenador y reprogramarlo cada vez que se cambia de aula o de router), se ha implementado la librería **WiFiManager.h** junto con **WebServer.h** y **DNSServer.h**.

En la práctica, cuando el ESP32 se enciende y no reconoce ninguna red WiFi cercana, detiene el envío de datos y levanta su propio punto de acceso WiFi llamado Maqueta_Guardia_ seguido de su dirección MAC.

El usuario solo tiene que conectarse a esa red desde su teléfono móvil, donde se abre automáticamente una ventana emergente (portal cautivo) para seleccionar el WiFi del lugar e introducir la clave. El chip guarda las credenciales en su memoria flash no volátil y se conecta automáticamente a partir de ese momento.

Protocolo de Transporte IoT (MQTT y Broker Mosquitto):

Para transmitir los datos desde el ESP32 hasta el ordenador en tiempo real y sin retardo, se utiliza el protocolo MQTT (Message Queuing Telemetry Transport).

A diferencia de las páginas web tradicionales (HTTP), MQTT es un protocolo ultraligero basado en una arquitectura de Publicación/Suscripción.

En la práctica, se ha instalado en el ordenador el servidor Eclipse Mosquitto (actuando como Broker o cartero central en la dirección IP local 192.168.1.64 por el puerto 1883).

El ESP32 actúa como Publicador (Publish), lee los sensores y dispara los números en bruto hacia "buzones" o canales específicos llamados Topics (por ejemplo, la_guardia/aceleracion/x o la_guardia/distancia). Mosquitto recibe estos paquetes al instante y los reparte a cualquier programa que esté escuchando, logrando una latencia prácticamente nula.

Procesamiento Matemático e Interfaz de Cabina (Node-RED):

Node-RED es una herramienta de programación visual basada en flujos (flow-based programming) ejecutada sobre un entorno Node.js en el ordenador local. Cumple dos funciones críticas en el proyecto:

Cerebro Matemático (Backend): Node-RED actúa como Suscriptor (Subscribe) conectándose al broker Mosquitto para escuchar todos los topics de la familia la_guardia/#.

Como el ESP32 solo envía fuerzas gravitatorias en bruto para no saturar su procesador, Node-RED recoge esos valores en nodos de función programados en JavaScript.

Allí aplica trigonometría avanzada (Math.atan2 e integración temporal) para convertir las fuerzas G en ángulos reales de Cabeceo (Pitch), Alabeo (Roll) y Guiñada (Yaw), gestionando además la puesta a cero mediante el botón de calibración (Tara).

Cuadro de Mandos y GPWS (Frontend / Dashboard): A través de los módulos gráficos node-red-dashboard, transforma los resultados matemáticos en una interfaz web interactiva que simula la cabina del avión.

Dibuja en pantalla un Horizonte Artificial animado mediante código HTML5/SVG que se inclina al mismo tiempo que la maqueta, actualiza los indicadores numéricos de telemetría y controla el sintetizador de voz del navegador (ui_audio) para emitir las alertas habladas de altitud ("50 metros", "100 metros") y de inclinación peligrosa ("Bank angle, bank angle").

4. Arquitectura y Funcionalidad del Sistema

El sistema de telemetría embarcada opera bajo una arquitectura distribuida donde la captura física de datos está desacoplada del procesamiento matemático pesado. Esto permite que el microcontrolador a bordo del avión trabaje a máxima velocidad sin bloqueos, delegando la trigonometría, la representación gráfica y la síntesis de voz al servidor central en Node-RED.

4.1. Esquema de Comunicaciones y Flujo de Datos (Arquitectura MQTT)

La comunicación entre la maqueta física y el panel de instrumentos se estructura en tres niveles jerárquicos conectados a través de la red local (IP del servidor: 192.168.1.64, puerto 1883):

[NIVEL 1: CAPTURA FÍSICA (Avión - ESP32)]

├— Sensor MPU6050 (I2C) → Lee vectores de fuerza G (X,Y,Z) y giro (X,Y,Z)

└— Sensor HC-SR04 (GPIO) → Mide rebote acústico y calcula distancia en cm

|

▼ (Publicación WiFi cada 100 ms - Tasa de 10 Hz)

[NIVEL 2: TRANSPORTE (Broker Mosquitto MQTT)]

├— Topic: la_guardia/aceleracion/x, y, z (Datos en bruto m/s^2)

├— Topic: la_guardia/giroscopio/x, y, z (Datos en bruto rad/s)

└— Topic: la_guardia/distancia (Altura en cm)

|

▼ (Suscripción comodín: "la_guardia/#")

[NIVEL 3: PROCESAMIENTO E INTERFAZ (Servidor Node-RED)]

├— Rama 1: Nodo "Cálculo Numérico" ← Botón "TARA" (Puesta a cero)

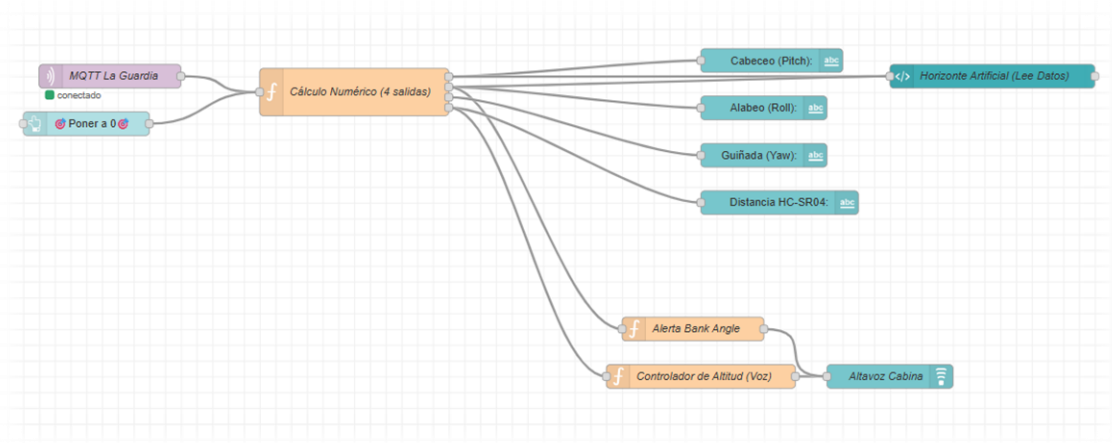
| ├→ Indicadores de Texto (Cabeceo, Alabeo, Guiñada y Distancia)

| ├→ Rama 3A: Controlador de Altitud (Filtro por rangos) —┐

| └→ Rama 3B: Alerta Bank Angle (Buffer de 10 muestras) → [Altavoz Cabina]

└— Rama 2: Nodo "Cálculo Horizonte" (Desacoplado)

└→ Plantilla HTML5/CSS/SVG (Horizonte Artificial Dinámico)



4.2. Telemetría Numérica, Trigonometría y Calibración (Botón de Tara)

El ESP32 no envía ángulos ya calculados, sino lecturas puras de los sensores. Cuando Node-RED recibe estos paquetes a través del nodo suscriptor MQTT, los envía al bloque lógico de Cálculo Numérico, el cual mantiene una memoria interna (context) con el estado de la aeronave y aplica las siguientes operaciones físicas:

Cálculo de Cabeceo (Pitch - Simulación de picado y encabritado) y Alabeo (Roll - Inclinación lateral):

Se obtienen descomponiendo el vector de la gravedad terrestre (9.81m/s^2) sobre los tres ejes del acelerómetro mediante la función arcotangente de dos argumentos (Math.atan2), convirtiendo el resultado de radianes a grados sexagesimales:

Cálculo de Guiñada (Yaw - Giro en rumbo sobre el eje vertical):

Al no variar la gravedad cuando el avión gira plano sobre sí mismo, la guiñada se calcula mediante integración numérica temporal de la velocidad angular del eje Z del giroscopio (g_z). El código mide el diferencial de tiempo exacto en segundos (\Δ t) entre cada mensaje recibido y acumula el desplazamiento angular:

Sistema de Calibración en Reposo (Botón de Tara):

Dado que es físicamente imposible colocar la maqueta sobre una mesa o montar el sensor en su interior con una precisión absoluta de 0.00, se ha habilitado un botón interactivo de Tara en el cuadro de mandos.

Al pulsarlo, Node-RED captura los ángulos de inclinación actuales en ese milisegundo y los guarda como variables de compensación (offsetPitch y offsetRoll), reiniciando además el acumulador de guiñada a cero.

A partir de ese instante, a todas las lecturas futuras se les resta esa tara inicial, permitiendo calibrar el avión en cualquier superficie u orientación.

Finalmente, un normalizador matemático corrige cualquier desbordamiento si el avión supera los 180º, evitando saltos numéricos erróneos.

4.3. Simulación Visual: El Horizonte Artificial Dinámico

Para que el piloto o usuario pueda interpretar la actitud del avión de un vistazo (especialmente durante maniobras rápidas de picado o alabeo), el panel incorpora un Horizonte Artificial aeronáutico construido a medida.

Para garantizar que la interfaz gráfica funcione con total fluidez y nunca se bloquee si se pulsa el botón de Tara o si hay picos de tráfico en la parte numérica, su cálculo se ha desacoplado en una rama paralela independiente.

Un segundo nodo matemático lee directamente los vectores ay y az del acelerómetro para extraer el ángulo de inclinación visual puro.

El ángulo calculado se inyecta en un nodo de plantilla web (ui_template) programado en HTML y CSS. Este bloque dibuja un instrumento circular dividido en dos mitades cromáticas (azul para el cielo y marrón para la tierra) con líneas de referencia de cabeceo (escalera de grados) y una silueta fija amarilla en el centro que representa las alas del avión. Mediante transformaciones dinámicas en cascada (transform: rotate(...) translateY(...)), la bola del horizonte rota y se desplaza verticalmente en la pantalla en tiempo real a medida que el usuario inclina o pica la maqueta física en el aire.

4.4. Sistema GPWS de Alertas por Voz (Altitud y Bank Angle)

Emulando los sistemas reales de advertencia de proximidad a tierra (Ground Proximity Warning System - GPWS) y de protección de envolvente de vuelo de aviones comerciales como el CRJ-900 involucrado en el accidente de LaGuardia, el servidor incorpora dos módulos de vigilancia autónoma conectados al sintetizador de voz del navegador (ui_audio):

1. Controlador de Altitud por Escalones (Radio Callouts):

Este nodo escucha continuamente los centímetros medidos por el sensor ultrasónico HC-SR04 y traduce la distancia física de la maqueta respecto a la mesa en altitudes aeronáuticas simuladas mediante ventanas de detección:

A los 300 cm Anuncia por voz: "300".

A los 200 cm Anuncia por voz: "200".

Y así sucesivamente

Para evitar que el sistema repita en bucle el mismo mensaje diez veces por segundo si el avión mantiene una altura constante dentro de una franja, el nodo implementa un filtro de memoria de estado (lastMsg), una vez cantada una altitud, el altavoz se silencia automáticamente para ese escalón y solo vuelve a armarse cuando el avión sale hacia una zona intermedia (por ejemplo, 8 cm o 13 cm) o entra en un escalón distinto.

2. Alerta de Inclinación Lateral Crítica (Bank Angle):

Si durante la aproximación final el avión sufre una inclinación excesiva de las alas, existe riesgo de pérdida de sustentación o impacto de la punta del ala contra la pista.

Para prevenirlo, un nodo específico monitoriza la salida matemática del eje de Alabeo buscando valores que igualen o superen el umbral crítico.

En lugar de disparar la alarma ante el primer dato elevado (lo que podría provocar falsas alarmas por un golpe brusco en la mesa o una vibración puntual del sensor), se ha programado un algoritmo de ventana deslizante (Buffer de memoria).

El nodo almacena en una matriz interna (history) los últimos 10 valores recibidos del sensor (equivalente a 1 segundo completo de vuelo a 10 Hz).

En cada nueva lectura, recorre la lista completa y cuenta cuántas muestras superan el valor crítico en valor absoluto (Math.abs).

Únicamente cuando 6 o más de los últimos 10 valores son críticos, el sistema confirma que se trata de una inclinación real y sostenida, activando el bloqueo anti-spam (isWarningActive) y enviando al altavoz de cabina la alerta urgente: "Bank angle, bank angle". Cuando el piloto corrige la posición del avión y la mayoría de las muestras bajan de ese ángulo crítico, la alarma se rearma en silencio.

5. Retos Técnicos y Soluciones Implementadas

Durante el desarrollo e integración del sistema de telemetría y aviónica embarcada, se presentaron diversos desafíos de índole física, matemática y de software.

-Problema 1: Ruido electrónico y saltos numéricos en los ángulos (Gimbal Lock y desbordamientos).

-Solución: Se implementó un filtrado en origen mediante la configuración del chip MPU6050 (ancho de banda del filtro paso bajo a 21 Hz) para mitigar las vibraciones mecánicas del entorno.

Además, en el lado de Node-RED se añadieron normalizadores matemáticos que restringen y limpian los valores de salida, evitando saltos erróneos en los límites.

-Problema 2: Bucle infinito y saturación del sintetizador de voz (Spam de alertas).

-Solución: Tanto en el sistema de altitud (Radio Callouts) como en el aviso de inclinación (Bank Angle), se programaron filtros de estado y buffers de memoria. En la altitud se estableció una memoria de escalones para que la voz no se repita estáticamente a la misma altura, y en el alabeo se implementó un algoritmo de ventana deslizante para evitar falsas alarmas por golpes puntuales en la mesa.

-Problema 3: Dependencia de una red WiFi estática y necesidad de reprogramación.

-Solución: Se eliminaron las credenciales hardcodeadas del código fuente de Arduino y se integró la librería WiFiManager. Esto permite que el ESP32 despliegue su propio portal cautivo si no detecta una red conocida, permitiendo configurar el acceso a internet de forma dinámica desde un teléfono móvil sin tocar el código.

-Problema 4: Falsas lecturas de inclinación causadas por tolerancias mecánicas.

-Solución: Al comprobar que las soldaduras inferiores del módulo GY-521 inclinaban ligeramente el circuito, se diseñó e imprimió en 3D un soporte específico con un cajeado inferior y tolerancias de ajuste milimétricas (+0.20 mm), garantizando que el sensor reposara de manera perfectamente plana y alineada con los ejes de la aeronave.

6. Conclusiones y Aprendizaje

6.1. Conclusiones del Proyecto

El desarrollo de este proyecto ha permitido constatar la enorme importancia de la seguridad operacional en la aviación civil, tomando como base analítica el trágico accidente del vuelo AC8646 en el aeropuerto LaGuardia.

El estudio de aquel suceso demostró que los accidentes aéreos raramente ocurren por un único fallo, sino por la confluencia de múltiples factores (como la falta de transpondedores en tierra, la saturación de los controladores y las limitaciones de los radares de superficie).

Aunque la planificación inicial abarcaba la recreación integral de la infraestructura terrestre y los sistemas de pistas, la delimitación práctica al entorno de la aeronave ha demostrado ser un acierto técnico.

Se ha logrado materializar con éxito una de las soluciones clave recomendadas tras el accidente: dotar al avión de un sistema de alerta autónomo en cabina independiente de la torre de control. El sistema embarcado de telemetría (ESP32, MPU6050 y HC-SR04) junto con el procesamiento en Node-RED y el sintetizador GPWS simula fielmente la aviónica de una aeronave comercial moderna.

6.2. Aprendizaje y Líneas Futuras

A nivel formativo, el trabajo ha supuesto un ejercicio multidisciplinar muy completo que ha combinado la ingeniería de diseño (modelado 3D de soportes), la programación de microcontroladores en C/C++, el uso de protocolos IoT ligeros (MQTT) y el desarrollo de interfaces lógicas y visuales en Node-RED.

Se ha aprendido a gestionar la latencia de red, la sincronización de flujos de datos en tiempo real y la resolución empírica de problemas de hardware y software.

Como línea de evolución futura, se plantea retomar la fase terrestre planteada en el proyecto teórico original.

Esto incluiría la sensorización del pavimento con fotorresistencias (LDR) y la integración de dichos sensores en el mismo broker MQTT, permitiendo que la maqueta del avión y la pista de aterrizaje interactúen de forma conjunta para detectar y prevenir incursiones en tiempo real en un entorno completamente automatizado

7. Anexos

Enlace GitHub: GITHUB_LaGuardiaAirport (códigos, modelos y manual de usuario)